

DIT5303 / DMT5751 Mobile Applications Development

DIT5308 / DMT5922 Embedded Systems & Hardware Interface

Project Report

Smart Piano

Member:

Lo Sek Kong	230154279	(ICT)
Pang Wang Chun	230088026	(ICT)
Chow Ho Yeung	250589584	(ICT)
Cheung Yat Yan	250543742	(MTI)
Li Wan Yeung	250276094	(MTI)

Content

The real-world question to solve	4
Role Allocation.....	5
Literature research.....	6
Technical challenge	8
App Code Description	10
Arduino.....	10
Key variables and modules	10
Main control flow	11
Communication protocol.....	12
LED and UI logic	12
Robustness and error handling.....	13
Testing and improvement suggestions	13
Android APP	13
Android Manifest.....	13
Purpose and role.....	13
Declared permissions	13
Activities and exported flags.....	14
FileProvider	14
Best practices and testing checklist.....	14
Layouts	15
Splash layout.....	15
Main layout	16
Menu layout	16
Menu page	17
Control layout	17

NOSound layout.....	18
Metronome layout.....	19
Recording list layout.....	20
file_paths.....	21
Java Classes	22
SplashActivity.....	22
MainActivity	22
BluetoothSocketHolder	25
MenuActivity	26
ControlActivity	27
NOSoundActivity.....	28
MetronomeActivity	28
RecordingListActivity	29
InternalAudioRecorder	29
Test case & result	31
Final product photo	31
Contribution.....	33
Co-operation Difficulty	34
Future work	35

The real-world question to solve

In our project Smart Piano, our target users are the beginners of learning how to play piano. We found that there are some real-world questions when they learn piano by themselves.

1. Don't know where to start when self-learning as there is no person to tutor

Piano beginners often face the problem of unclear guidance during self-directed learning. Without professional instruction, they typically do not receive fundamental lessons in music theory, such as how to read staff notation or numbered musical notation. Consequently, they may have no way of knowing whether they are correctly interpreting the content of a musical score during their practice.

Furthermore, they must find learning materials on their own, most commonly by searching the internet. However, the online space is flooded with numerous different versions of tutorials and teaching methods. On their own, they lack the ability to filter through these options and determine which version they should follow. In many cases, after being exposed to multiple conflicting approaches, their understanding of how to play the piano becomes even more confused and fragmented. This overload of information, instead of providing clarity, often leads to paralysis, leaving them unsure of where or how to begin their learning journey effectively. This highlights a critical gap between the availability of online resources and a beginner's ability to navigate and apply them constructively.

2. Lack of visual rhythm assistant tools

The lack of visible rhythmic aids presents another significant challenge for piano beginners. As mentioned in the previous section, learners often lack foundational music theory knowledge. This deficit means they may struggle to independently grasp the correct tempo for a piece or accurately assess whether the rhythm they are using is right.

While they might purchase physical metronomes from various sources, these devices often come with different designs and control mechanisms. For a novice, the function of each button, dial, or slider is not immediately intuitive or obvious at first glance. Without clear, visual guidance, operating these tools correctly becomes a hurdle, further complicating their effort to establish and maintain proper timing during practice.

3. Lack of practice records and review mechanisms to know the process of learning

A lack of practice records makes meaningful review and progress tracking exceptionally difficult. Piano beginners often need to rely on evaluations of their past practice sessions to make informed adjustments to their subsequent playing. Typically operating on a limited budget, many learners are unlikely to purchase a dedicated metronome and instead opt to use metronome applications on their smartphones.

However, a significant practical problem arises here: such metronome apps generally cannot operate simultaneously with a phone video recording application. This creates a frustrating limitation. Without a professional teacher providing a steady tempo or to record and analyze their errors, learners find themselves unable to both keep time with a metronome and create a video recording that could later be used to diagnose their playing issues.

Role Allocation

Lo Sek Kong from ICT, role in the project is project manager. Responsibilities for creating project planning, managing teams, tracking progress, and ensuring the final output meets quality expectations.

Pang Wang Chun from ICT, role in the project is chief software officer. Responsible for Arduino programming and hardware development, including programming testing.

Chow Ho Yeung from ICT, role is Software Officer. Responsibilities for developing mobile applications and configuring Arduino programs.

Cheung Yat Yan from MTI, role is marketing officer. Responsibilities for researching the problems of usage and collecting advice from users.

Li Wan Yeung from MTI, role is design officer. Responsibilities for appearance design and APP UI.

Literature research

1. Don't know where to start when self-learning

"A survey of 200 beginning piano students revealed that 78% practice without specific goals, 65% repeat entire pieces rather than problematic sections, and 92% cannot identify their own technical errors during practice." - American Music Teacher Magazine, June/July 2021 issue

"Beginning pianists read music at approximately 20-30% of their reading speed for text, with particular difficulty in tracking both clefs simultaneously and anticipating upcoming notes." - "Improving Sight-Reading Skills" by Paul Harris

"Students frequently confuse basic terms: 60% cannot correctly distinguish between 'legato' and 'staccato' after three months, and 75% misunderstand time signatures beyond 4/4." - "Teaching Piano for the First Time" by Karen Harrington

2. Lack of visual rhythm assistant tools

"Beginning piano students often struggle with maintaining consistent tempo, particularly when shifting attention between hands. The cognitive load of reading notation while executing motor commands disrupts internal timing mechanisms." - "The Psychology of Music" (3rd Edition) by Diana Deutsch

"In our study of 50 piano beginners (ages 6-10), 78% could not accurately identify whether they were playing rhythms correctly without visual feedback. When provided with a simple visual metronome showing beat divisions, accuracy in self-assessment improved by 62%. Students consistently reported: 'I thought I was playing evenly, but the lights showed I was speeding up during difficult passages.'" - McPherson, G. E., & Gabrielsson, A. (2002). From sound to sign. In R. Parncutt & G. E. McPherson (Eds.), *The Science and Psychology of Music Performance* (pp. 99-116). Oxford University Press.

"Observation of beginning piano students revealed a persistent discrepancy between their perceived rhythm accuracy and actual performance. In one documented case, a student playing

'Minuet in G' believed they maintained steady tempo throughout, but metronome analysis showed fluctuations of up to 30 BPM during section transitions. The student remarked: 'It all sounds the same to me—I can't tell when I speed up or slow down.'" - Duke, R. A., & Simmons, A. L. (2006). The nature of expertise: Narrative descriptions of 19 common elements observed in the lessons of three renowned artist-teachers. *Bulletin of the Council for Research in Music Education*, 170, 7-19.

3. Lack of practice records and review mechanisms to know the process of learning

"Beginning piano students often struggle to track their progress systematically. In our study of 45 novice pianists, 78% reported having no consistent method for recording practice sessions. One participant noted, 'I practice every day but can't tell if I'm actually improving. I wish I had a way to look back and see what I worked on last month.'" - *Journal of Research in Music Education*, 2013, Volume 61, Issue 1

User u/beginnerpianist88 posted: "Been playing for 6 months but have zero evidence of improvement. My teacher says I'm getting better but I can't see it myself. How do you guys track what you've learned?"

Top response with 245 upvotes: "I use a spreadsheet now. Before that, I was just practicing blindly with no direction or record of what I'd accomplished." - r/piano subreddit discussion thread "Tracking Progress as a Beginner", May 2022

"Our research indicates that beginners who maintained practice journals showed 40% greater retention of learned material compared to those who didn't. However, only 15% of participants in the control group naturally developed record-keeping habits without intervention." - *Proceedings of the International Conference on Music Perception and Cognition*, 2018

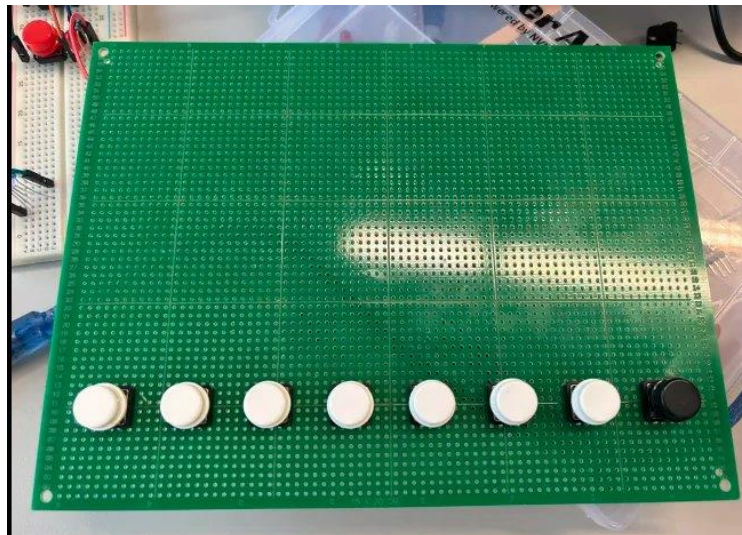
Technical challenge

Technical challenge 1:

Our initial goal was to perfectly replicate a 14-key piano (7 black keys + 7 white keys), with each key having a corresponding LED. In addition, octave up and octave down buttons were included, requiring a total of 30 output pins. However, the number of pins we needed exceeded the maximum pin limit of the ESP32.

Solution:

We redesigned the black keys as a single button. By holding down the black key button and pressing the corresponding white key, the system will play the sound of the associated black key.



Technical challenge 2:

We initially considered using common-anode RGB LEDs to indicate the black and white keys. For example, when a white key is pressed on the phone, the corresponding piano LED would light up in green; When a black key is pressed, the corresponding LED would light up in blue. This design would allow users to clearly distinguish between black and white key indications. However, during the testing phase, we encountered two major difficulties:

1. The number of required pins exceeded the ESP32's available pin count, making it impossible to achieve the intended multi-color effect.

2. Due to the large number of buttons and LEDs, the system experienced insufficient power supply, which caused two LEDs to fail to light up during testing.

Solution: We adopted standard single-color LEDs instead. The indication of black and white keys is expressed through brightness levels. White keys are shown with full brightness, while black keys are shown with half brightness.

Technical challenge 3:

We use **snprintf** to build complete message strings and send them with a single **SerialBT.write** call, which is faster and avoids splitting into multiple Bluetooth packets. USB Serial (**Serial.print**) output are minimized to prevent blocking when no computer is connected and only enabled in debug mode. Key detection will still include debounce, but once a press is confirmed the message would be sent immediately without waiting for other steps. Mobile command handling also changed to single write operations instead of multiple prints. Every key press reliably sends a **PRESSED...** message, each ending with its own newline, so the Android parser can process each message instantly.

App Code Description

Arduino

[ESP32_Piano.ino](#) is the firmware that interfaces physical piano hardware with the Android client over Bluetooth SPP. It reads seven white-key inputs and three control inputs (black key, octave up, octave down), drives seven LEDs with PWM, updates an OLED display, and sends/receives compact commands to/from the phone. The firmware emphasizes **low latency**, **debounced inputs**, and **non-blocking Bluetooth writes**.

Key variables and modules

- **BluetoothSerial SerialBT** for SPP communications.

```
void btWriteString(const char* s) {  
    if (!SerialBT.hasClient()) return;  
    size_t len = strlen(s);  
    if (len == 0) return;  
    SerialBT.write((const uint8_t*)s, (size_t)len);  
}
```

```
// === Setup ===  
void setup() {  
    if (DEBUG_SERIAL) Serial.begin(115200);  
    SerialBT.begin("ESP32_PIANO");
```

- OLED display using `Adafruit_SSD1306` and `updateOctaveDisplay()` to show octave and black-key state.

```
void updateOctaveDisplay() {
    int octaveNumber;
    if (octaveFlag == -1) octaveNumber = 3;
    else if (octaveFlag == 0) octaveNumber = 4;
    else octaveNumber = 5;

    // 清畫面並顯示
    display.clearDisplay();
    display.setTextSize(2);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(0, 0);
    display.print("Octave:");
    display.println(octaveNumber);

    display.setTextSize(1);
    display.setCursor(0, 24);
    if (blackPressed) display.print("Black mode: ON");
    else display.print("Black mode: OFF");

    display.display();
}
```

- Pin arrays: `LED_PINS[7]`, `BUTTON_PINS[7]`; single pins for `BTN_BLACK_PIN`, `BTN_OCT_UP_PIN`, `BTN_OCT_DOWN_PIN`.
- State: `blackPressed` (bool), `octaveFlag` (int: -1,0,1), `ledBrightness[ ]`, `ledMode[ ]`.
- Debounce: per-key `lastReading[ ]`, `stableState[ ]`, `lastDebounceTime[ ]`; similar variables for control buttons.
- Constants: `DEBOUNCE_MS` (50 ms), `DEBUG_SERIAL` toggle.

Main control flow

1. **setup**: initialize Bluetooth (`SerialBT.begin`), OLED (`display.begin`), configure LED pins as outputs and button pins as `INPUT_PULLUP`, call `allOff()` to clear LEDs.
2. **loop**: three responsibilities repeated continuously:
 - a. **Process incoming Bluetooth bytes**: `while (SerialBT.available())` reads one char at a time and calls `handleIncomingChar(c)` to act immediately on single-character commands from the phone.
 - b. **Debounce and handle control buttons**: black key and octave up/down are

debounced; on state change the firmware updates **blackPressed/octaveFlag**, updates OLED, and sends **ONBLACK/OFFBLACK** or **OctN** messages.

- c. **Debounce and handle seven white keys:** on confirmed press compose **PRESSEDn;B:x;O:y\n** and send via **SerialBT.write** (single write call). Optionally turn LEDs off depending on current LED mode and black state.

```
if ((millis() - lastDebounceTime[i]) > DEBOUNCE_MS) {
  if (stableState[i] == HIGH && reading == LOW) { // 確定按下
    // 立即組訊息並送出 (一次 write)
    int bflag = blackPressed ? 1 : 0;
    int oflag = octaveFlag;
    char buf[48];
    int n = snprintf(buf, sizeof(buf), "PRESSED%d;B:%d;O:%d\n", i + 1, bflag, oflag);
    if (n > 0 && SerialBT.hasClient()) {
      SerialBT.write((const uint8_t*)buf, (size_t)n);
    }
    if (DEBUG_SERIAL) {
      Serial.print("Sent: ");
      Serial.println(buf);
    }
  }
}
```

Communication protocol

- **Outgoing text messages:** **PRESSEDn;B:x;O:y\n**, **ONn\n**, **OFF\n**, **ONBLACK\n**, **OFFBLACK\n**, **OctN\n**.
- **Incoming single-character commands:** **'1'..'7'** to light a key, **'0'** for all off, **'b'/'B'** for black on/off, **'u'/'d'** for octave up/down.
- **Write strategy:** use **SerialBT.write((const uint8_t*)buf, len)** to perform single non-blocking writes and reduce overhead.

LED and UI logic

- **setLed(idx, brightness, mode)** centralizes PWM writes and local state updates.
- **allOff()** resets LEDs and modes.
- **Turn-off logic** after a **PRESSED** event checks **ledMode** and **ledBrightness** to decide whether to extinguish the LED.

```
bool shouldTurnOff = false;
if (blackPressed && ledMode[i] == 1 && ledBrightness[i] == 40) {
  shouldTurnOff = true;
} else if (!blackPressed && ledMode[i] == 0 && ledBrightness[i] == 255) {
  shouldTurnOff = true;
}
if (shouldTurnOff) {
  setLed(i, 0, 0);
}
```

Robustness and error handling

- Check `SerialBT.hasClient()` before writing.
- Continue operation if OLED initialization fails.
- Minimal blocking: short `delay(2)` only to yield CPU.

Testing and improvement suggestions

- **Test cases:** rapid repeated presses, simultaneous presses, Bluetooth disconnect/reconnect, malformed incoming commands.
- **Improvements:** consider a compact binary packet mode with sequence numbers or checksums for critical flows; optionally use hardware interrupts for keys if lower latency is required.

Android APP

Android Manifest

Purpose and role

[AndroidManifest.xml](#) declares app metadata, permissions, activities, features, and the `FileProvider` used for sharing recordings. It is the authoritative manifest for runtime permission expectations and exported components.

Declared permissions

- **Bluetooth:** `BLUETOOTH`, `BLUETOOTH_ADMIN`, `BLUETOOTH_SCAN`, `BLUETOOTH_CONNECT`, `BLUETOOTH_ADVERTISE`.
- **Location:** `ACCESS_COARSE_LOCATION`, `ACCESS_FINE_LOCATION` (legacy compatibility for older Android versions).
- **Audio and storage:** `RECORD_AUDIO`, `READ_EXTERNAL_STORAGE`, `WRITE_EXTERNAL_STORAGE`. **Note:** Many of these require runtime requests on modern

Android; the manifest only declares them.

Activities and exported flags

- **SplashActivity**: exported, launcher (MAIN + LAUNCHER).
- **MainActivity**: exported.
- **MenuActivity**, **ControlActivity**, **NOSoundActivity**, **MetronomeActivity**, **RecordingListActivity**: not exported (internal navigation). **Security note**: exported activities should validate incoming intents if they accept external data.

FileProvider

- Declares `androidx.core.content.FileProvider` with `authorities="${applicationId}.fileprovider"`, `exported="false"`, and `grantUriPermissions="true"`.
- References `@xml/file_paths` to restrict shareable directories (e.g., **Music/Recordings**). This enables secure sharing of audio files without exposing raw file paths.

Best practices and testing checklist

- **Runtime permission flows**: implement and test on Android 8 through 13 for Bluetooth and recording behaviors.
- **Android 12+**: ensure correct handling of **BLUETOOTH_SCAN** and **BLUETOOTH_CONNECT** runtime flows.
- **Storage**: prefer `getExternalFilesDir()` and **FileProvider** to avoid broad storage permissions on modern Android.
- **Edge cases**: test behavior when permissions are denied permanently, when Bluetooth is disabled, and when MediaProjection is unavailable.

```

▼<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <!-- Bluetooth / location / storage / audio permissions
      注意：部分權限需在執行時請求（尤其 Android 12+ 的藍牙權限） -->
  <uses-permission android:name="android.permission.BLUETOOTH"/>
  <uses-permission android:name="android.permission.BLUETOOTH_ADMIN"/>
  <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
  <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
  <uses-permission android:name="android.permission.BLUETOOTH_SCAN"/>
  <uses-permission android:name="android.permission.BLUETOOTH_CONNECT"/>
  <uses-permission android:name="android.permission.BLUETOOTH_ADVERTISE"/>
  <uses-permission android:name="android.permission.RECORD_AUDIO"/>
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
  <uses-feature android:name="android.hardware.bluetooth" android:required="false"/>
  ▼<application android:allowBackup="true" android:label="ESP32 Piano" android:icon="@mipmap/ic_launcher" android:roundIcon="@mipmap/ic_launcher_round"
    android:theme="@style/AppTheme">
    ▼<activity android:name=".SplashActivity" android:exported="true" android:theme="@style/AppTheme">
      ▼<intent-filter>
        <action android:name="android.intent.action.MAIN"/>
        <category android:name="android.intent.category.LAUNCHER"/>
      </intent-filter>
    </activity>
    <activity android:name=".MainActivity" android:exported="true" android:theme="@style/AppTheme"/>
    <activity android:name=".MenuActivity" android:exported="false" android:label="Menu"/>
    <activity android:name=".ControlActivity" android:exported="false" android:label="Control"/>
    <activity android:name=".NOSoundActivity" android:exported="false" android:label="NOSound"/>
    <activity android:name=".MetronomeActivity" android:exported="false" android:label="Metronome"/>
    <activity android:name=".RecordingListActivity" android:exported="false" android:label="Recordings"/>
    ▼<provider android:name="androidx.core.content.FileProvider" android:authorities="${applicationId}.fileprovider" android:exported="false"
      android:grantUriPermissions="true">
      <meta-data android:name="android.support.FILE_PROVIDER_PATHS" android:resource="@xml/file_paths"/>
    </provider>
  </application>
</manifest>

```

Layouts

Splash layout

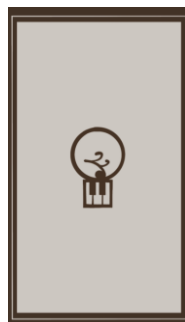
- File: [activity_splash.xml](#)
- Key elements: `ImageView splash_bg` for background, `ImageView splash_logo` centered.
- Purpose: brief branded splash screen; `SplashActivity` transitions to `MainActivity` after a short delay.

LOADING page

```

▼<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android" android:id="@+id/splash_root" android:layout_width="match_parent" android:layout_height="match_parent"
  android:background="@android:color/white">
  <!-- 背景圖：可用 drawable 或單色 -->
  <ImageView android:id="@+id/splash_bg" android:layout_width="match_parent" android:layout_height="match_parent" android:scaleType="centerCrop"
    android:src="@drawable/background_" android:contentDescription="splash background"/>
  <!-- 中央 logo -->
  <ImageView android:id="@+id/splash_logo" android:layout_width="200dp" android:layout_height="200dp" android:layout_gravity="center" android:src="@drawable/logo"
    android:contentDescription="logo"/>
</FrameLayout>

```



Main layout

- File: [activity_main.xml](#)
- Key elements: TextView tvStatus, EditText etDeviceName, Button btnScan, Button btnConnect, Button btnNext, Button btnRequestPermissions.
- Purpose: entry point for Bluetooth discovery/connection and permission requests; shows connection status and allows navigation to the menu.

Bluetooth connection page

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/main_root"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="#BEB9B2"
    android:backgroundTint="#BEB9B2"
    android:padding="16dp">
    <TextView
        android:id="@+id/tvStatus"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="狀態"
        android:textSize="16sp"
        android:padding="8dp"/>
    <EditText
        android:id="@+id/etDeviceName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Device name"/>
    <Button
        android:id="@+id/btnScan"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Scan"/>
    <Button
        android:id="@+id/btnConnect"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Connect"/>
    <Button
        android:id="@+id/btnNext"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Next (Menu)"/>
    <!-- 新增：一次性請求所有權限的按鈕 -->
    <View
        android:layout_width="match_parent"
        android:layout_height="12dp"/>
    <Button
        android:id="@+id/btnRequestPermissions"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="授權所有必要權限"
        android:backgroundTint="#537A77"
        android:textColor="#BEB9B2"/>
</LinearLayout>
```



Menu layout

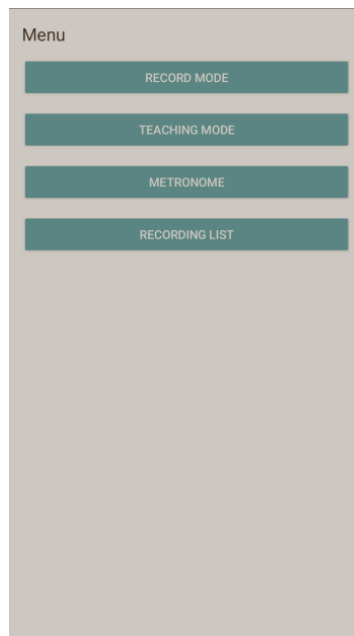
- File: [activity_menu.xml](#)
- Key elements: Buttons btnControl, btnNOSound, btnMetronome,

btnRecordingList.

- **Purpose:** navigation hub to launch the app's main functional activities.

Menu page

```
▼ <ScrollView xmlns:android="http://schemas.android.com/apk/res/android" android:id="@+id/menu_root" android:layout_width="match_parent" android:layout_height="match_parent"
  android:padding="16dp" android:background="#BEB9B2" android:backgroundTint="#BEB9B2">
  ▼ <LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content" android:orientation="vertical">
    <TextView android:id="@+id/tvMenuTitle" android:layout_width="match_parent" android:layout_height="wrap_content" android:text="Menu" android:textSize="20sp"
      android:textColor="#332417" android:paddingBottom="12dp"/>
    <!-- Control -->
    <Button android:id="@+id/btnControl" android:layout_width="match_parent" android:layout_height="wrap_content" android:backgroundTint="#537A77" android:textColor="#BEB9B2"
      android:text="Record Mode"/>
    <View android:layout_width="match_parent" android:layout_height="12dp"/>
    <!-- NOsound -->
    <Button android:id="@+id/btnNOsound" android:layout_width="match_parent" android:layout_height="wrap_content" android:backgroundTint="#537A77" android:textColor="#BEB9B2"
      android:text="Teaching Mode"/>
    <View android:layout_width="match_parent" android:layout_height="12dp"/>
    <!-- Metronome -->
    <Button android:id="@+id/btnMetronome" android:layout_width="match_parent" android:layout_height="wrap_content" android:backgroundTint="#537A77"
      android:textColor="#BEB9B2" android:text="Metronome"/>
    <View android:layout_width="match_parent" android:layout_height="12dp"/>
    <!-- Recording List -->
    <Button android:id="@+id/btnRecordingList" android:layout_width="match_parent" android:layout_height="wrap_content" android:backgroundTint="#537A77"
      android:textColor="#BEB9B2" android:text="Recording List"/>
  </LinearLayout>
</ScrollView>
```

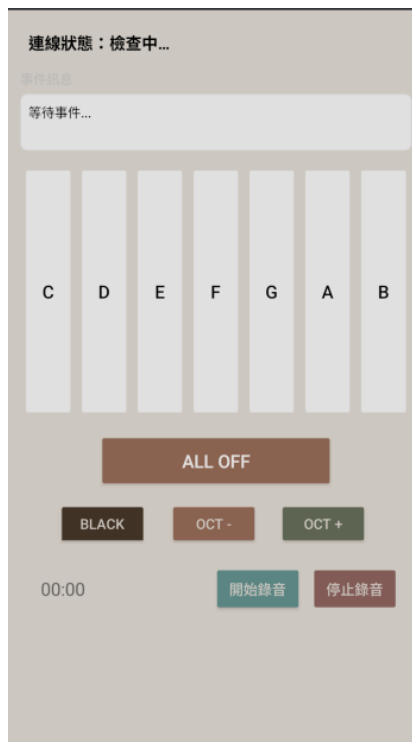


Control layout

- File: [activity_control.xml](#)
- Key elements: TextView tvConn, TextView tvEvents, LinearLayout whiteKeysRow containing btn1..btn7, Button btnAllOff, ToggleButton btnBlack, Button btnOctDown, Button btnOctUp, recording block with tvRecordTimer, btnStartRecord, btnStopRecord.
- **Purpose:** full control UI for live interaction with ESP32, local playback, and recording management.

Record Mode page

```
<?xml version="1.0" encoding="utf-8" android:namespace="http://schemas.android.com/apk/res/android" android:label="@string/app_name">
<android.support.design.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android" xmlns:app="http://schemas.android.com/apk/res-auto" android:id="@+id/root" android:layout_width="match_parent"
    android:layout_height="match_parent" android:padding="12dp" android:background="#BEE982" android:backgroundTint="#BEE982">
    <!-- Top status and title -->
    <TextView android:id="@+id/tvConn" android:layout_width="8dp" android:layout_height="wrap_content" android:text="連線狀態：檢查中..." android:textSize="16sp" android:textStyle="bold" android:textColor="@android:color/black"
        android:padding="8dp" android:contentDescription="Connection status" app:layout_constraintTop_toTopOf="parent" app:layout_constraintStart_toStartOf="parent" app:layout_constraintEnd_toEndOf="parent"/>
    <!-- Event label -->
    <TextView android:id="@+id/tvEventsLabel" android:layout_width="wrap_content" android:layout_height="wrap_content" android:text="事件訊息" android:textSize="13sp" android:textColor="@android:color/darker_gray"
        android:paddingTop="6dp" android:contentDescription="Event information label" app:layout_constraintTop_toBottomOf="parent" app:layout_constraintStart_toStartOf="parent"/>
    <!-- Event area (compact) -->
    <TextView android:id="@+id/tvEvents" android:layout_width="8dp" android:layout_height="56dp" android:layout_marginTop="6dp" android:padding="8dp" android:background="@drawable/event_area_bg" android:text="等待事件..."
        android:textSize="13sp" android:textColor="@android:color/black" android:ellipsize="end" android:maxLines="3" android:contentDescription="Event log" app:layout_constraintTop_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent" app:layout_constraintEnd_toEndOf="parent"/>
    <!-- White keys row (C D E F G A B) -->
    <LinearLayout android:id="@+id/whiteKeysRow" android:layout_width="8dp" android:layout_height="8dp" android:orientation="horizontal" android:gravity="center" android:layout_marginTop="12dp"
        app:layout_constraintTop_toBottomOf="parent" app:layout_constraintStart_toStartOf="parent" app:layout_constraintEnd_toEndOf="parent" app:layout_constraintHeight_default="percent" app:layout_constraintHeight_percent="0.36">
        <Button android:id="@+id/btnC" style="@style/KeyButton.White" android:text="C" android:textAllCaps="false" android:contentDescription="Key C"/>
        <Button android:id="@+id/btnD" style="@style/KeyButton.White" android:text="D" android:textAllCaps="false" android:contentDescription="Key D"/>
        <Button android:id="@+id/btnE" style="@style/KeyButton.White" android:text="E" android:textAllCaps="false" android:contentDescription="Key E"/>
        <Button android:id="@+id/btnF" style="@style/KeyButton.White" android:text="F" android:textAllCaps="false" android:contentDescription="Key F"/>
        <Button android:id="@+id/btnG" style="@style/KeyButton.White" android:text="G" android:textAllCaps="false" android:contentDescription="Key G"/>
        <Button android:id="@+id/btnA" style="@style/KeyButton.White" android:text="A" android:textAllCaps="false" android:contentDescription="Key A"/>
        <Button android:id="@+id/btnB" style="@style/KeyButton.White" android:text="B" android:textAllCaps="false" android:contentDescription="Key B"/>
    </LinearLayout>
    <!-- Large ALL OFF button centered below keys -->
    <Button android:id="@+id/btnAllOff" android:layout_width="8dp" android:layout_height="56dp" android:text="ALL OFF" android:textAllCaps="true" android:textSize="18sp" android:backgroundTint="#7E5948"
        android:textColor="@android:color/white" android:contentDescription="All off" android:layout_marginTop="12dp" app:layout_constraintTop_toBottomOf="parent" app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" app:layout_constraintWidth_percent="0.6"/>
    <!-- Controls row: BLACK, OCT -, OCT + -->
    <LinearLayout android:id="@+id/controlRow" android:layout_width="8dp" android:layout_height="wrap_content" android:orientation="horizontal" android:gravity="center" android:layout_marginTop="12dp"
        app:layout_constraintTop_toBottomOf="parent" app:layout_constraintStart_toStartOf="parent" app:layout_constraintEnd_toEndOf="parent">
        <ToggleButton android:id="@+id/btnBlack" android:layout_width="wrap_content" android:layout_height="44dp" android:text="BLACK ON" android:textOff="BLACK" android:backgroundTint="#332417" android:textColor="#BEE982"
            android:contentDescription="Black key toggle"/>
        <Space android:layout_width="16dp" android:layout_height="wrap_content"/>
        <Button android:id="@+id/btnOctDown" style="@style/ControlButton" android:text="OCT -" android:backgroundTint="#7E5948" android:textColor="#BEE982" android:contentDescription="Octave down"/>
        <Space android:layout_width="8dp" android:layout_height="wrap_content"/>
        <Button android:id="@+id/btnOctUp" style="@style/ControlButton" android:text="OCT +" android:backgroundTint="#55584A" android:textColor="#BEE982" android:contentDescription="Octave up"/>
    </LinearLayout>
    <!-- 錄音區塊 -->
    <LinearLayout android:id="@+id/record_block" android:layout_width="8dp" android:layout_height="wrap_content" android:orientation="horizontal" android:padding="12dp" android:gravity="center_vertical" android:layout_marginTop="6dp"
        app:layout_constraintTop_toBottomOf="parent" app:layout_constraintStart_toStartOf="parent" app:layout_constraintEnd_toEndOf="parent">
        <TextView android:id="@+id/tvRecordTime" android:layout_width="8dp" android:layout_height="11dp" android:layout_weight="1" android:layout_height="wrap_content" android:text="00:00" android:textSize="18sp" android:padding="8dp"/>
        <Button android:id="@+id/btnStartRecord" android:layout_width="wrap_content" android:layout_height="wrap_content" android:text="開始錄音" android:backgroundTint="#537A77" android:textColor="#BEE982"/>
        <Space android:layout_width="8dp" android:layout_height="wrap_content"/>
        <Button android:id="@+id/btnStopRecord" android:layout_width="wrap_content" android:layout_height="wrap_content" android:text="停止錄音" android:enabled="false" android:backgroundTint="#77534F" android:textColor="#BEE982"/>
    </LinearLayout>
</android.support.design.widget.ConstraintLayout>
```

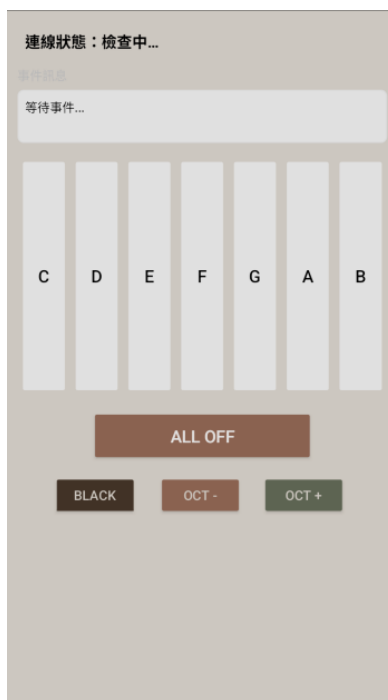


NOSound layout

- File: [activity_nosound.xml](#)
- Key elements: similar to Control layout: **tvConn**, **tvEvents**, seven key buttons, **btnAllOff**, **btnBlack**, octave controls.
- Purpose: teaching mode where the phone plays sounds locally and synchronizes visual state with the ESP32.

Teaching Mode page

```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android" xmlns:app="http://schemas.android.com/apk/res-auto" android:id="@+id/root" android:layout_width="match_parent"
android:layout_height="match_parent" android:padding="12dp" android:background="#BEE992" android:backgroundTint="#BEE992">
<!-- Top status and title -->
<TextView android:id="@+id/tvConn" android:layout_width="0dp" android:layout_height="wrap_content" android:text="連線狀態：檢查中..." android:textSize="16sp" android:textStyle="bold" android:textColor="@android:color/black"
android:padding="8dp" android:contentDescription="Connection status" app:layout_constraintTop_toTopOf="parent" app:layout_constraintStart_toStartOf="parent" app:layout_constraintEnd_toEndOf="parent"/>
<!-- Event label -->
<TextView android:id="@+id/tvEventsLabel" android:layout_width="wrap_content" android:layout_height="wrap_content" android:text="事件訊息" android:textSize="13sp" android:textColor="@android:color/darker_gray"
android:paddingTop="6dp" android:contentDescription="Event information label" app:layout_constraintTop_toBottomOf="@+id/tvConn" app:layout_constraintStart_toStartOf="parent"/>
<!-- Event area (compact) -->
<TextView android:id="@+id/tvEvents" android:layout_width="0dp" android:layout_height="56dp" android:layout_marginTop="6dp" android:padding="8dp" android:background="@drawable/event_area_bg" android:text="等待事件..."
android:textSize="13sp" android:textColor="@android:color/black" android:ellipsize="end" android:maxLines="3" android:contentDescription="Event log" app:layout_constraintTop_toBottomOf="@+id/tvEventsLabel"
app:layout_constraintStart_toStartOf="parent" app:layout_constraintEnd_toEndOf="parent"/>
<!-- White keys row (C D E F G A B) -->
<LinearLayout android:id="@+id/whiteKeysRow" android:layout_width="0dp" android:layout_height="0dp" android:orientation="horizontal" android:gravity="center" android:layout_marginTop="12dp"
app:layout_constraintTop_toBottomOf="@+id/tvEvents" app:layout_constraintStart_toStartOf="parent" app:layout_constraintEnd_toEndOf="parent" app:layout_constraintHeight_default="percent" app:layout_constraintHeight_percent="0.36">
<Button android:id="@+id/btnC" style="@style/KeyButton.White" android:text="C" android:textAllCaps="false" android:contentDescription="Key C"/>
<Button android:id="@+id/btnD" style="@style/KeyButton.White" android:text="D" android:textAllCaps="false" android:contentDescription="Key D"/>
<Button android:id="@+id/btnE" style="@style/KeyButton.White" android:text="E" android:textAllCaps="false" android:contentDescription="Key E"/>
<Button android:id="@+id/btnF" style="@style/KeyButton.White" android:text="F" android:textAllCaps="false" android:contentDescription="Key F"/>
<Button android:id="@+id/btnG" style="@style/KeyButton.White" android:text="G" android:textAllCaps="false" android:contentDescription="Key G"/>
<Button android:id="@+id/btnA" style="@style/KeyButton.White" android:text="A" android:textAllCaps="false" android:contentDescription="Key A"/>
<Button android:id="@+id/btnB" style="@style/KeyButton.White" android:text="B" android:textAllCaps="false" android:contentDescription="Key B"/>
</LinearLayout>
<!-- Large ALL OFF button centered below keys -->
<Button android:id="@+id/btnAllOff" android:layout_width="0dp" android:layout_height="56dp" android:text="ALL OFF" android:textAllCaps="true" android:textSize="18sp" android:backgroundTint="#7E948"
android:textColor="@android:color/white" android:contentDescription="All off" android:layout_marginTop="12dp" app:layout_constraintTop_toBottomOf="@+id/whiteKeysRow" app:layout_constraintStart_toStartOf="parent"
app:layout_constraintEnd_toEndOf="parent" app:layout_constraintWidth_percent="0.6"/>
<!-- Controls row: BLACK, OCT -, OCT + -->
<LinearLayout android:id="@+id/controlRow" android:layout_width="0dp" android:layout_height="wrap_content" android:orientation="horizontal" android:gravity="center" android:layout_marginTop="12dp"
app:layout_constraintTop_toBottomOf="@+id/btnAllOff" app:layout_constraintStart_toStartOf="parent" app:layout_constraintEnd_toEndOf="parent">
<ToggleButton android:id="@+id/btnBlack" android:layout_width="wrap_content" android:layout_height="44dp" android:textOn="BLACK ON" android:textOff="BLACK" android:backgroundTint="#332417" android:textColor="#BEE992"
android:contentDescription="Black key toggle"/>
<Space android:layout_width="16dp" android:layout_height="wrap_content"/>
<Button android:id="@+id/btnOctDown" style="@style/ControlButton" android:text="OCT -" android:backgroundTint="#7E948" android:textColor="#BEE992" android:contentDescription="Octave down"/>
<Space android:layout_width="8dp" android:layout_height="wrap_content"/>
<Button android:id="@+id/btnOctUp" style="@style/ControlButton" android:text="OCT +" android:backgroundTint="#55584A" android:textColor="#BEE992" android:contentDescription="Octave up"/>
</LinearLayout>
</androidx.constraintlayout.widget.ConstraintLayout>
```



Metronome layout

- File: [activity metronome.xml](#)
- Key elements: TextView tvBpm, TextView tvBeats, SeekBar seekBpm, buttons for BPM and beats adjustments, btnStartStop, btnTap, and LinearLayout beatGrid.
- Purpose: metronome UI with BPM control, beat-per-bar selection, visual beat grid, and tap-tempo support.

Metronome page

```
▼<ScrollView xmlns:android="http://schemas.android.com/apk/res/android" android:layout_width="match_parent" android:layout_height="match_parent" android:backgroundTint="#BEB9B2" android:background="#BEB9B2"
android:padding="12dp">
    ▼<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content" android:orientation="vertical">
        <TextView android:id="@+id/tvHeader" android:layout_width="match_parent" android:layout_height="wrap_content" android:text="節拍器" android:textSize="20sp" android:textStyle="bold" android:paddingBottom="8dp"/>
        ▼<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content" android:orientation="horizontal" android:backgroundTint="#BEB9B2" android:padding="12dp" android:elevation="2dp">
            ▼<LinearLayout android:layout_width="0dp" android:layout_weight="1" android:layout_height="wrap_content" android:orientation="vertical">
                <TextView android:id="@+id/tvBpmLabel" android:layout_width="wrap_content" android:layout_height="wrap_content" android:text="速度" android:textSize="14sp" android:textColor="#666666"/>
                <TextView android:id="@+id/tvBpm" android:layout_width="wrap_content" android:layout_height="wrap_content" android:text="100" android:textSize="36sp" android:textStyle="bold" android:layout_marginTop="4dp"/>
            </LinearLayout>
            ▼<LinearLayout android:layout_width="wrap_content" android:layout_height="match_parent" android:orientation="vertical" android:gravity="center_vertical">
                <Button android:id="@+id/btnDecBpm" android:layout_width="48dp" android:layout_height="48dp" android:text="-"/>
                <Button android:id="@+id/btnIncBpm" android:layout_width="48dp" android:layout_height="48dp" android:text="+"/>
            </LinearLayout>
        </LinearLayout>
        <SeekBar android:id="@+id/seekBpm" android:layout_width="match_parent" android:layout_height="wrap_content" android:layout_marginTop="10dp" android:max="214"/>
        ▼<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content" android:orientation="horizontal" android:gravity="center_vertical" android:layout_marginTop="12dp">
            <TextView android:layout_width="wrap_content" android:layout_height="wrap_content" android:text="每小節拍數" android:textSize="14sp" android:textColor="#666666"/>
            <View android:layout_width="0dp" android:layout_weight="1" android:layout_height="1dp"/>
            <Button android:id="@+id/btnDecBeats" android:layout_width="44dp" android:layout_height="48dp" android:text="-"/>
            <TextView android:id="@+id/tvBeats" android:layout_width="48dp" android:layout_height="wrap_content" android:text="4" android:textSize="20sp" android:gravity="center" android:layout_marginLeft="8dp"
            android:layout_marginRight="8dp"/>
            <Button android:id="@+id/btnIncBeats" android:layout_width="44dp" android:layout_height="48dp" android:text="+"/>
        </LinearLayout>
        ▼<LinearLayout android:id="@+id/beatGridContainer" android:layout_width="match_parent" android:layout_height="wrap_content" android:orientation="vertical" android:layout_marginTop="14dp">
            <TextView android:layout_width="wrap_content" android:layout_height="wrap_content" android:text="拍子視覺" android:textSize="14sp" android:textColor="#666666" android:paddingBottom="6dp"/>
            <LinearLayout android:id="@+id/beatGrid" android:layout_width="match_parent" android:layout_height="100dp" android:orientation="horizontal" android:background="@android:color/transparent">
            </LinearLayout>
        </LinearLayout>
        ▼<LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content" android:orientation="horizontal" android:gravity="center" android:layout_marginTop="18dp">
            <Button android:id="@+id/btnStartStop" android:layout_width="80dp" android:layout_height="48dp" android:text="開始"/>
            <View android:layout_width="12dp" android:layout_height="wrap_content"/>
            <Button android:id="@+id/btnTap" android:layout_width="80dp" android:layout_height="48dp" android:text="拍子速度"/>
        </LinearLayout>
        <TextView android:layout_width="match_parent" android:layout_height="wrap_content" android:text="提示：BPM 範圍 30-244。按下開始後會播放拍子聲，並在每小節第一拍發送同步標記（若已連線）。" android:textSize="12sp"
        android:textColor="#777777" android:layout_marginTop="12dp" android:paddingBottom="18dp"/>
    </ScrollView>
```



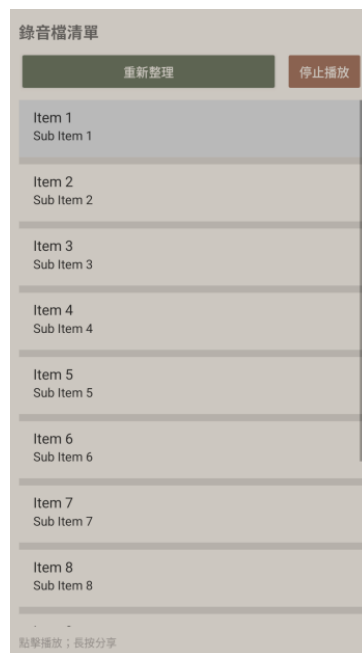
Recording list layout

- File: [activity_recording_list.xml](#)
- Key elements: TextView tvRecordListTitle, Button btnRefreshList, Button btnStopPlay, ListView lvRecordings, TextView tvRecordListHint.

- **Purpose:** list and manage recorded audio files; supports play and share via long-press.

Recording List page

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android" android:id="@+id/record_list_root" android:layout_width="match_parent"
    android:layout_height="match_parent" android:orientation="vertical" android:padding="12dp" android:background="#8E8982" android:backgroundTint="#8E8982">
    <TextView android:id="@+id/tvRecordListTitle" android:layout_width="match_parent" android:layout_height="wrap_content" android:text="錄音檔清單" android:textSize="18sp"
        android:textStyle="bold" android:paddingBottom="8dp"/>
    <LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content" android:orientation="horizontal" android:paddingBottom="8dp">
        <Button android:id="@+id/btnRefreshList" android:layout_width="0dp" android:layout_height="1" android:layout_weight="1" android:layout_height="48dp" android:text="重新整理"
            android:backgroundTint="#555B4A" android:textColor="#8E8982"/>
        <Space android:layout_width="8dp" android:layout_height="wrap_content"/>
        <Button android:id="@+id/btnStopPlay" android:layout_width="wrap_content" android:layout_height="48dp" android:text="停止播放" android:enabled="false"
            android:backgroundTint="#7E5948" android:textColor="#8E8982"/>
    </LinearLayout>
    <ListView android:id="@+id/lvRecordings" android:layout_width="match_parent" android:layout_height="0dp" android:layout_weight="1" android:dividerHeight="8dp"
        android:listSelector="@android:color/darker_gray"/>
    <TextView android:id="@+id/tvRecordListHint" android:layout_width="match_parent" android:layout_height="wrap_content" android:text="點擊播放；長按分享"
        android:textSize="12sp" android:paddingTop="8dp" android:textColor="#87827E"/>
</LinearLayout>
```



file_paths

- File: res/xml/file_paths.xml
- Key elements: `<external-files-path name="external_music_recordings" path="Music/Recordings" />` and `<files-path name="internal_files" path="." />`.
- **Purpose:** restrict FileProvider-accessible directories to app-controlled locations for secure sharing.

```
<?xml version="1.0" encoding="utf-8"?>
<paths xmlns:android="http://schemas.android.com/apk/res/android">
    <external-files-path name="external_music_recordings" path="Music/Recordings" />
    <files-path name="internal_files" path="." />
</paths>
```

Java Classes

SplashActivity

- [SplashActivity.java](#)
- **Responsibility:** show splash UI and transition to **MainActivity** after a short delay.
- **Key methods:** **onCreate** sets full-screen flags and **Handler.postDelayed** to start **MainActivity**.
- **Notes:** avoid heavy initialization here to keep startup snappy.

Loading page into Bluetooth connect page

```
package com.example.esp32spp;

import android.content.Intent;
import android.os.Bundle;
import android.os.Handler;
import androidx.appcompat.app.AppCompatActivity;
import android.view.WindowManager;

/**
 * SplashActivity
 * - 顯示啟動畫面若干毫秒後轉到 MainActivity
 * - 放在 src/main/java/com/example/esp32spp/
 */
public class SplashActivity extends AppCompatActivity {
    private static final long SPLASH_MS = 1200L; // 顯示時間，毫秒

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        // 可選：全螢幕顯示（若使用 Toolbar 或狀態列控制請調整）
        getWindow().setFlags(WindowManager.LayoutParams.FLAG_FULLSCREEN,
            WindowManager.LayoutParams.FLAG_FULLSCREEN);

        setContentView(R.layout.activity_splash);

        // 延遲後啟動 MainActivity（或你想要的啟動 Activity）
        new Handler().postDelayed(() -> {
            Intent it = new Intent(SplashActivity.this, MainActivity.class);
            startActivity(it);
            finish();
        }, SPLASH_MS);
    }
}
```

MainActivity

- [MainActivity.java](#)
- **Responsibility:** manage Bluetooth adapter, list paired devices, initiate SPP connection, and handle permission prompts.
- **Key flows:**

- **Auto-fill:** `attemptAutoFillEsp32Name()` searches bonded devices for `ESP32_PIANO`.
- **Scan:** `showPairedDevicesDialog()` lists paired devices; requests `BLUETOOTH_SCAN` on Android 12+.
- **Connect:** `startPersistentConnect(nameOrAddr)` creates an RFCOMM socket to the SPP UUID, connects, stores socket in `BluetoothSocketHolder`, and spawns a simple read/write thread for logging and keepalive pings.
- **Permissions:** request `RECORD_AUDIO` and `BLUETOOTH_SCAN` on demand; handle `onRequestPermissionsResult`.
- **Threading:** `connectThread` for connection attempts; separate read thread for socket IO.
- **Testing:** verify behavior when Bluetooth is off, when device not found, and when permissions are denied.

call `BluetoothSocketHolder` by using SPP UUID to connect Arduino

```
public class MainActivity extends AppCompatActivity {
    private static final String TAG = "MainActivity";
    private static final UUID SPP_UUID = UUID.fromString("00001101-0000-1000-8000-00805F9B34FB");
    private static final String DEFAULT_TARGET_NAME = "ESP32_PIANO";
    private static final int REQ_BLUETOOTH_SCAN = 5001;
```

```
// Connect 按鈕：使用 etDeviceName 的名稱或 address 嘗試連線
btnConnect.setOnClickListener(new android.view.View.OnClickListener() {
    @Override
    public void onClick(android.view.View v) {
        if (btAdapter == null) return;
        if (!btAdapter.isEnabled()) {
            Intent enableBt = new Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE);
            startActivity(enableBt);
            tvStatus.setText("請開啟藍牙後再按 Connect");
            return;
        }
        String nameOrAddr = etDeviceName.getText().toString().trim();
        if (nameOrAddr.isEmpty()) nameOrAddr = DEFAULT_TARGET_NAME;
        startPersistentConnect(nameOrAddr);
    }
});

// Next 按鈕：進入 Menu (需先連線)
btnNext.setOnClickListener(new android.view.View.OnClickListener() {
    @Override
    public void onClick(android.view.View v) {
        BluetoothSocket s = BluetoothSocketHolder.getSocket();
        if (s == null || !s.isConnected()) {
            tvStatus.setText("尚未連線，無法進入控制頁面");
            return;
        }
        startActivity(new Intent(MainActivity.this, MenuActivity.class));
    }
});
```

```

// 處理權限回呼 (BLUETOOTH_SCAN 或其他)
@Override
public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions,
                                       @NonNull int[] grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);
    if (requestCode == REQ_BLUETOOTH_SCAN) {
        if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
            showPairedDevicesDialog();
        } else {
            Toast.makeText(this, "藍牙掃描權限未授予，無法列出裝置", Toast.LENGTH_SHORT).show();
            new AlertDialog.Builder(this)
                .setTitle("權限未授予")
                .setMessage("若要列出已配對裝置，請在系統設定中授予藍牙掃描權限。")
                .setPositiveButton("前往設定", new DialogInterface.OnClickListener() {
                    @Override
                    public void onClick(DialogInterface dialog, int which) {
                        Intent intent = new Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS);
                        intent.setData(Uri.parse("package:" + getPackageName()));
                        startActivity(intent);
                    }
                })
                .setNegativeButton("取消", null)
                .show();
        }
    } else if (requestCode == 301) { // RECORD_AUDIO legacy callback
        if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
            Toast.makeText(this, "錄音權限已授予", Toast.LENGTH_SHORT).show();
        } else {
            Toast.makeText(this, "錄音權限未授予", Toast.LENGTH_SHORT).show();
        }
    }
}
}

```


BluetoothSocketHolder

- [BluetoothSocketHolder.java](#)
- **Responsibility:** simple synchronized static holder for a **BluetoothSocket** so multiple Activities can share the same connected socket.
- **API:** `setSocket(BluetoothSocket)`, `getSocket()`, `clear()`.
- **Notes:** lightweight; consider replacing with a connection manager for reconnection logic and centralized lifecycle control.

```
package com.example.esp32spp;

import android.bluetooth.BluetoothSocket;

/**
 * Simple holder for a BluetoothSocket so multiple Activities can access it.
 * In your app you may have a more robust connection manager.
 */
public class BluetoothSocketHolder {
    private static BluetoothSocket socket;

    public static synchronized void setSocket(BluetoothSocket s) {
        socket = s;
    }

    public static synchronized BluetoothSocket getSocket() {
        return socket;
    }

    public static synchronized void clear() {
        socket = null;
    }
}
```

MenuActivity

- [MenuActivity.java](#)
- **Responsibility:** bind menu buttons and launch corresponding Activities.
- **Key behavior:** defensive checks for null views and **try/catch** around **startActivity** to surface errors via **Toast** and logs.

Menu page control button page to other page: record mode, teaching mode, metronome, recording list

```
btnControl.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        try {
            startActivity(new Intent(MenuActivity.this, ControlActivity.class));
        } catch (Exception e) {
            Log.e(TAG, "Failed to start ControlActivity", e);
            Toast.makeText(MenuActivity.this, "無法開啟 Control", Toast.LENGTH_SHORT).show();
        }
    }
});

btnNOSound.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        try {
            startActivity(new Intent(MenuActivity.this, NOSoundActivity.class));
        } catch (Exception e) {
            Log.e(TAG, "Failed to start NOSoundActivity", e);
            Toast.makeText(MenuActivity.this, "無法開啟 NOSound", Toast.LENGTH_SHORT).show();
        }
    }
});

btnMetronome.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        try {
            startActivity(new Intent(MenuActivity.this, MetronomeActivity.class));
        } catch (Exception e) {
            Log.e(TAG, "Failed to start MetronomeActivity", e);
            Toast.makeText(MenuActivity.this, "無法開啟 Metronome", Toast.LENGTH_SHORT).show();
        }
    }
});

btnRecordingList.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        try {
            startActivity(new Intent(MenuActivity.this, RecordingListActivity.class));
        } catch (Exception e) {
            Log.e(TAG, "Failed to start RecordingListActivity", e);
            Toast.makeText(MenuActivity.this, "無法開啟 Recording List", Toast.LENGTH_SHORT).show();
        }
    }
});
```

ControlActivity

- [ControlActivity.java](#)
- **Responsibility:** primary control surface — parse incoming messages (text and binary), play audio samples, manage recording, and update UI state.
- **Key components:**
 - **SoundPool:** multi-octave sample arrays **resOctaves**, **soundIdsOctaves**, **loadedOctaves**; pre-warm on load complete to reduce first-play latency; queue pending plays until loaded.
 - **Bluetooth I/O:** obtain socket from **BluetoothSocketHolder**, open **in/out**, start **listenThread** to parse both ASCII messages and binary packets with header **0x7E**. **handleBinaryPacket** decodes event codes and payload bitfields for low-latency events.
 - **Recording:** integrates **InternalAudioRecorder** (API 29+) for system audio capture via MediaProjection and a microphone fallback using **MediaRecorder**. Manages recording file creation, UI timer, and permission flow.
 - **UI:** **setButtonLit** and **clearAllButtonLit** manage visual key states; **tvEvents** and **tvConn** show status.
- **Threading and cleanup:** background listen thread, **uiHandler** for main-thread updates; **onDestroy** stops threads, closes streams, and releases **SoundPool**. Socket is not closed here to allow reuse by other Activities.

Record mode

```
// Binary event codes
private static final int BIN_HDR = 0x7E;
private static final int BIN_EVT_PRESSED = 0x01;
private static final int BIN_EVT_ON = 0x02;
private static final int BIN_EVT_OFF = 0x03;
private static final int BIN_EVT_ONBLACK = 0x04;
private static final int BIN_EVT_OFFBLACK = 0x05;
private static final int BIN_EVT_OCT = 0x06;

private static class PlayRequest {
    int octaveIndex;
    int noteIndex;
    PlayRequest(int o, int n) { octaveIndex = o; noteIndex = n;
}
```

NOSoundActivity

- [NOSoundActivity.java](#)
- **Responsibility:** teaching mode variant of ControlActivity focused on local playback and UI synchronization without relying on physical audio output.
- **Key differences:**
 - **playNowImmediate:** allows playback directly from the receiving thread (SoundPool is thread-safe for play calls).
 - **Local state sync:** updates **localOctaveFlag** on **Oct** messages and reflects black-key state.
 - **Lifecycle:** in **onDestroy** this Activity closes the socket and calls **BluetoothSocketHolder.clear()** to explicitly release the shared socket (design choice to avoid resource leaks).
- **Testing:** ensure local playback timing, UI sync, and resource release are correct.

Teaching mode

MetronomeActivity

- [MetronomeActivity.java](#)
- **Responsibility:** metronome with BPM control, beat-per-bar selection, visual beat grid, click sound playback, and optional Bluetooth markers for synchronization.
- **Key components:**
 - **SoundPool:** loads **click** and optional note samples (**note1..note37**) if present.
 - **Timing:** **Handler.postDelayed** with **tickRunnable** drives ticks; **intervalMs()** computes period from BPM.
 - **UI:** **seekBpm** for BPM, **buildBeatGrid()** to create beat blocks, **highlightBeat()** to show current beat.
 - **Bluetooth marker:** **sendBluetoothMarker** sends short tags like "M", "S", "X" to the ESP32 on bar start, start, and stop.
- **Testing:** verify BPM accuracy, tap-tempo responsiveness, click latency, and marker arrival timing on the ESP32.

Metronome page do metronome function

RecordingListActivity

- [RecordingListActivity.java](#)
- **Responsibility:** list, play, and share recorded audio files stored under the app's external files directory.
- **Key components:**
 - **File management:** `getRecordingsDir()` uses `getExternalFilesDir(Environment.DIRECTORY_MUSIC)/Recordings`.
 - **List refresh:** `refreshList()` filters audio extensions, sorts by last modified, and populates `ListView`.
 - **Playback:** `MediaPlayer` used to play selected files; `stopPlayback()` handles cleanup.
 - **Sharing:** `FileProvider.getUriForFile()` to create a content URI and `Intent.ACTION_SEND` with `FLAG_GRANT_READ_URI_PERMISSION`.
- **Testing:** ensure FileProvider URIs are accessible to recipients, playback works across devices, and long-press share grants read permission.

Recording list page find the recorded m4a sound file

InternalAudioRecorder

[InternalAudioRecorder.java](#)

Responsibility: capture system audio via `MediaProjection` + `AudioPlaybackCapture`, encode PCM to AAC with `MediaCodec`, and write an `.m4a` file via `MediaMuxer`. Requires API 29+.

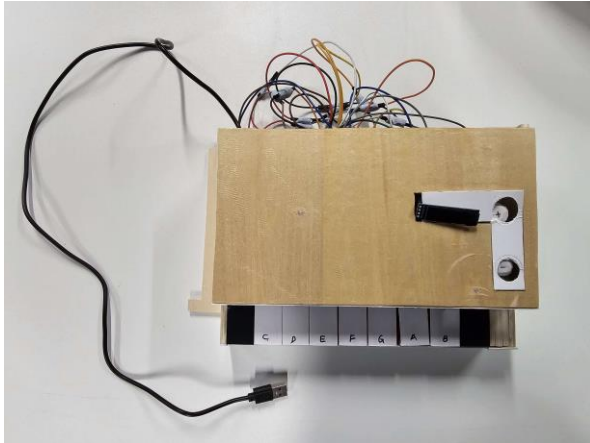
Workflow: `createScreenCaptureIntent()` provides the projection intent; `startProjection(resultCode, data, outFile)` creates `MediaProjection`, configures `AudioRecord` with an `AudioPlaybackCaptureConfiguration`, configures `MediaCodec` encoder, starts `captureThread`, reads PCM, queues input buffers, drains encoder output, and writes encoded frames to `MediaMuxer`. `stop()` stops capture, flushes encoder, stops muxer, releases resources, and stops `MediaProjection`. **Important details:** uses stereo 44.1 kHz PCM, AAC LC profile, `bitRate = 128000`, large read buffers, and starts muxer only after encoder output format is available. `capturing` is volatile and `captureThread.join()` ensures orderly shutdown. **Testing:** permission denied, projection revoked mid capture, encoder

failure, low memory, and abrupt termination. Recommend exposing callbacks for progress and errors and making sample rate/bitrate configurable.

Recording function active in record mode

Test case & result

Final product photo



Video 1: Breadboard Testing

Video Link: <https://www.youtube.com/watch?v=CCWs3RSqiHE>

	Test Case	Test Result
1	Scan and connect ESP32	Successfully connected
2	Press all white key buttons on app	The phone play the white key sound and LEDs on breadboard light up
3	Press all white key buttons on breadboard	All LEDs on breadboard turn off and the phone play the white key sounds
4	Press all black key buttons on app	The phone play the black key sound and LEDs on breadboard light up at half brightness
5	Press all black key buttons on breadboard	All LEDs on breadboard turn off and the phone play the black key sounds
6	Press the octave up/down button on the phone and breadboard and press all white/black key buttons on on the phone and breadboard	The phone correctly plays the black and white key sounds of the corresponding octave
7	Press ALL OFF button on app	All LEDs on breadboard turn off

Video 2: Smart Piano Testing

Video Link: <https://www.youtube.com/watch?v=PsN5mQfunqY>

	Test Case	Test Result
1	Scan and connect ESP32	Successfully connected
2	Press each button in the MENU one by one	Successfully switched to the corresponding interface
3	Press the Start Recording and Stop Recording buttons in the Record Mode interface	Successfully recorded and saved the file in M4A format to the phone
4	Play, pause, or share the selected recording from the Recording List interface	All functions are operating successfully
5	Press all white key buttons on the Teaching Mode interface.	The phone play the white key sounds and LEDs on piano light up.
6	Press all white key buttons on piano	All LEDs on piano turn off and the phone play the white key sounds
7	Press all black key buttons on the Teaching Mode interface.	The phone play the black key sound and LEDs on piano light up at half brightness.
8	Press all black key buttons on piano	All LEDs on piano turn off and the phone play the black key sounds
9	Press the octave up/down button on both phone and piano, press all white/ black key buttons on both phone and piano.	The phone correctly plays the black and white key sounds of the corresponding octave
10	Press ALL OFF button on Teaching Mode interface.	All LEDs on piano turn off
11	OLED monitor	The OLED monitor successfully displays the current octave value and whether a black key has been pressed
12	Testing all function on Metronome interface	All functions of the metronome are operating correctly

Contribution

Lo Sek Kong, main contribution to the project is supporting teammates and offering help whenever needed. On the hardware side, he contributed by purchasing Arduino components, designing circuit diagrams, performing a small portion of the soldering work. assisting teammates during soldering work, and performing hardware testing. On the software side, he helped teammates by typing AI prompts for vibe coding as he is the only member capable of typing in Chinese within the team, and conducting software testing.

Pang Wang Chun, main contribution to the project is taking the lead in hardware work, including purchasing Arduino components, performing most of the soldering work, and performing hardware testing. On the software side, he contributes by providing musical knowledge to support the design of the application.

CHOW Ho Yeung, main contribution to the project taking the lead in software work, including developing the mobile application with Android Studio, programming in Arduino IDE, providing support for logical concepts, and conducting software testing. On the hardware side, he assisted by purchasing Arduino components, supporting teammates during soldering work, and performing hardware testing.

Cheung Yat Yan, main contribution to the project is taking the lead in piano product design, including the exterior design, dimensional measurements, and material selection. On the software side, she contributes by designing the mobile application' s logo, loading screen, the UI/UX layout, structure, and color scheme.

Co-operation Difficulty

The core reason this entire collaboration derailed is Li Wang Yeung, whose contribution to the project can be summarized in one word: nothing. He didn't just miss both consultations — he vanished like a ghost with zero explanation, leaving the team to deal with the wreckage he created. His absence didn't just cause an information gap; it created a black hole of incompetence that sucked up everyone's time and energy.

While the rest of the team worked, he contributed absolutely no presence, no effort, no responsibility. Instead, he left behind a mess so big the group had to rebuild the entire workflow from the ground up just to compensate for his negligence. It's almost impressive how someone can participate so little yet cause so much damage.

His behavior shows a level of disregard that borders on spectacular laziness — a complete refusal to show up, to communicate, or to act like a functioning member of a team. It's not just unprofessional; it's actively sabotaging.

The team isn't here to babysit him, spoon feed him, or drag his dead weight across the finish line.

Going forward, the process needs to be rewired from scratch to prevent him from derailing the project again. If he wants to be part of the team, he needs to actually show up and do the work. If not, he should step aside instead of being a walking obstacle the rest of the group has to constantly clean up after.

Future work

In future development, we aim to enhance the system by first stabilizing the Bluetooth connection through the adoption of Bluetooth Low Energy (BLE) technology, ensuring more reliable and efficient communication. To support additional hardware features, we plan to expand the number of available pins, which will allow integration of new modules such as a dedicated speaker for direct audio playback and build a full octave piano.

Building on this, we intend to implement SD card support so that audio files can be stored and played independently through the speaker, increasing convenience and system autonomy. Another important direction is enabling MIDI file reading, which will allow automatic LED control based on musical signals, providing intuitive visual guidance for learners. Beyond playback, we also envision adding composition capabilities, where our application can return button signals and record them into MIDI files, transforming the system into a creative platform for music-making.

In addition to these functions, we plan to introduce musical effects such as sustain and harmony: sustain will simulate the behavior of a real piano by allowing notes to continue playing while keys are pressed and stopping only when released, while harmony will enable polyphonic performance by allowing multiple buttons to be pressed simultaneously and triggering multiple audio files at once. Together, these effects will provide a more authentic and immersive playing experience, bridging the gap between digital simulation and acoustic performance.

Reference

Deutsch, D. (2013). *The psychology of music* (3rd ed.). Academic Press.

Duke, R. A., & Simmons, A. L. (2006). The nature of expertise: Narrative descriptions of 19 common elements observed in the lessons of three renowned artist-teachers. *Bulletin of the Council for Research in Music Education*, 170, 7-19. <https://doi.org/10.2307/40319347>

Harris, P. (2021, June/July). Improving sight-reading skills. *American Music Teacher Magazine*, 70(6), 18-21.

Harrington, K. (2021, June/July). Teaching piano for the first time. *American Music Teacher Magazine*, 70(6), 22-25.

McPherson, G. E., & Gabrielsson, A. (2002). From sound to sign. In R. Parncutt & G. E. McPherson (Eds.), *The science and psychology of music performance* (pp. 99-116). Oxford University Press.

Reddit user beginnerpianist88. (2022, May). Tracking progress as a beginner [Online forum post]. Reddit. https://www.reddit.com/r/piano/comments/xxxxxxxxx/tracking_progress_as_a_beginner/

Simons, J. R., & Williams, K. A. (2013). Practice habits and self-regulation in beginning instrumentalists. *Journal of Research in Music Education*, 61(1), 87-102. <https://doi.org/10.1177/0022429412474826>

Various authors. (2018, July). *Proceedings of the International Conference on Music Perception and Cognition*. International Conference on Music Perception and Cognition.